DEEP LEARNING

Course Code: 22CAC08N

Panigrahi Srikanth
Assistant Professor
Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)

Chaitanya Bharathi Institute of Technology (Autonomous)
Hyderabad

Academic Year 2026–27

Course Objectives

Upon successful completion of this course, students will be able to:

1. Understand the fundamental concepts, history, and mathematical foundations of Deep Learning.
2. Learn optimization techniques including Gradient Descent, Backpropagation, and Regularization methods.
3. Design and implement Convolutional Neural Networks (CNNs) for image classification and computer vision tasks.
4. Understand sequence modeling using Recurrent Neural Networks (RNNs), LSTM, and GRU architectures.
5. Explore advanced Deep Learning models such as Transformers, Attention Mechanisms, and Generative Adversarial Networks (GANs).

Course Outcomes

At the end of this course, students will be able to:

1. Explain the fundamental concepts, architectures, and applications of Deep Learning.
2. Apply optimization algorithms and regularization techniques to train deep neural networks effectively.
3. Design and implement Convolutional Neural Networks (CNNs) for image classification and computer vision applications.
4. Analyze sequential data using Recurrent Neural Networks (RNNs), LSTM, and GRU architectures.
5. Develop intelligent solutions using advanced Deep Learning models such as Transformers, Attention Mechanisms, and GANs.

Skills Acquired

Deep Learning Fundamentals • Computer Vision • Sequence Modeling • Model Optimization • Modern AI Architectures

Unit I – Introduction to Deep Learning

Topics Covered

- Evolution of Artificial Intelligence, Machine Learning and Deep Learning
- Biological Neuron vs Artificial Neuron
- Perceptron and Multi-Layer Perceptron (MLP)
- Activation Functions
- Loss Functions
- Forward Propagation and Backpropagation

Learning Outcome

Understand the fundamentals of deep neural networks and the mathematical principles behind their learning process.

Unit II – Optimization and Regularization

Topics Covered

- Gradient Descent Optimization
- Stochastic & Mini-batch Gradient Descent
- Momentum and Nesterov Optimization
- Adam Optimizer
- Weight Initialization
- Batch Normalization
- Dropout Regularization

Learning Outcome

Apply optimization algorithms to improve convergence and prevent overfitting in deep neural networks.

Unit III – Convolutional Neural Networks

Topics Covered

- CNN Architecture
- Convolution Operation
- Pooling Layers
- Padding and Stride
- Feature Extraction
- Popular CNN Models: LeNet, AlexNet, VGGNet, GoogLeNet, and ResNet

Learning Outcome

Design CNN architectures for image classification and computer vision applications.

Unit IV – Recurrent Neural Networks

Topics Covered

- Recurrent Neural Networks (RNN)
- Vanishing Gradient Problem
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)
- Encoder–Decoder Architecture
- Sequence-to-Sequence Learning
- Attention Mechanism

Learning Outcome

Develop sequence models for language processing, speech recognition, and time-series prediction.

Unit V – Advanced Deep Learning Models

Topics Covered

- Transformer Architecture
- Self-Attention Mechanism
- Vision Transformers (ViT)
- BERT and GPT Models
- Generative Adversarial Networks (GANs)
- Deep Learning Applications

Learning Outcome

Understand modern deep learning architectures and their real-world AI applications.

Introduction to AI, Machine Learning and Deep Learning

Artificial Intelligence (AI)

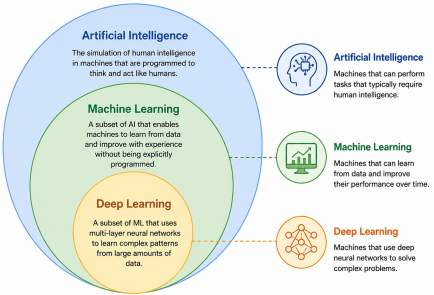
Developing intelligent systems that can perform tasks requiring human intelligence such as learning, reasoning, planning, and decision making.

Machine Learning (ML)

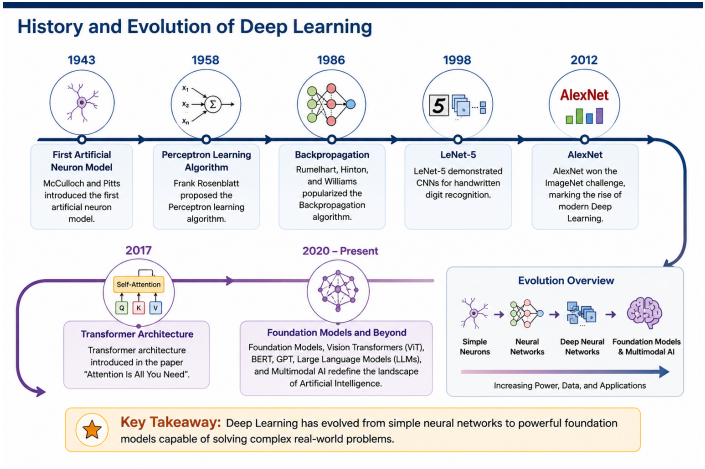
A subset of AI that enables computers to learn patterns from data without being explicitly programmed.

Deep Learning (DL)

A specialized subset of Machine Learning that uses multi-layer neural networks to automatically learn complex representations from large datasets.



History and Evolution of Deep Learning



Key Takeaway

Deep Learning has evolved from simple artificial neurons to powerful foundation models that drive modern Artificial Intelligence.

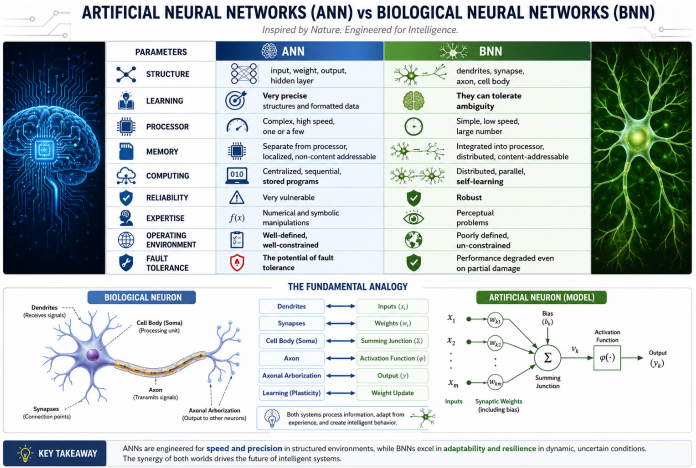
History and Evolution of Deep Learning

Year	Milestone
1943	McCulloch and Pitts introduced the first artificial neuron model.
1958	Frank Rosenblatt proposed the Perceptron learning algorithm.
1986	Rumelhart, Hinton, and Williams popularized Backpropagation.
1998	LeNet-5 demonstrated CNNs for handwritten digit recognition.
2012	AlexNet won the ImageNet challenge, marking the rise of modern Deep Learning.
2017	Transformer architecture introduced ("Attention Is All You Need").
2020–Present	Foundation Models, Vision Transformers (ViT), BERT, GPT, Large Language Models (LLMs), and Multimodal AI.

Key Takeaway

Deep Learning has evolved from simple neural networks to powerful foundation models capable of solving complex real-world problems.

Biological Neuron vs Artificial Neuron



Key Takeaway

Artificial Neural Networks (ANNs) are computational models inspired by Biological Neural Networks (BNNs). While BNNs are highly adaptive and fault tolerant, ANNs excel in speed, precision, and solving complex computational problems.

Biological Neuron vs Artificial Neuron

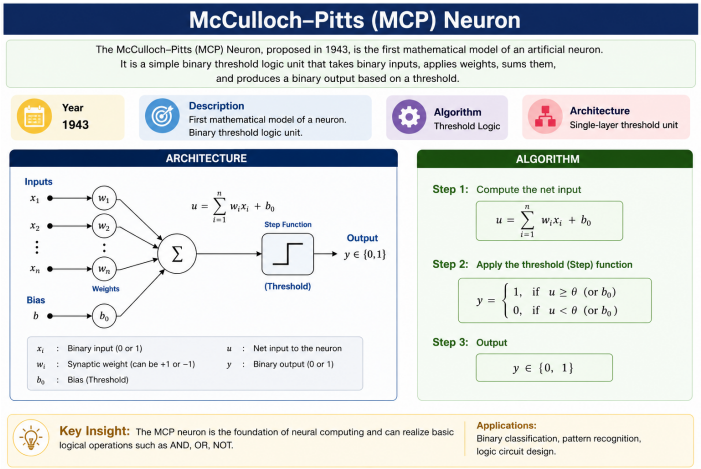
Biological Neuron

- Basic unit of the human nervous system
- Receives signals through dendrites
- Cell body processes information
- Axon transmits output signals
- Synapses connect neurons

Artificial Neuron

- Receives input features
- Each input has an associated weight
- Computes weighted sum
- Applies an activation function
- Produces an output value

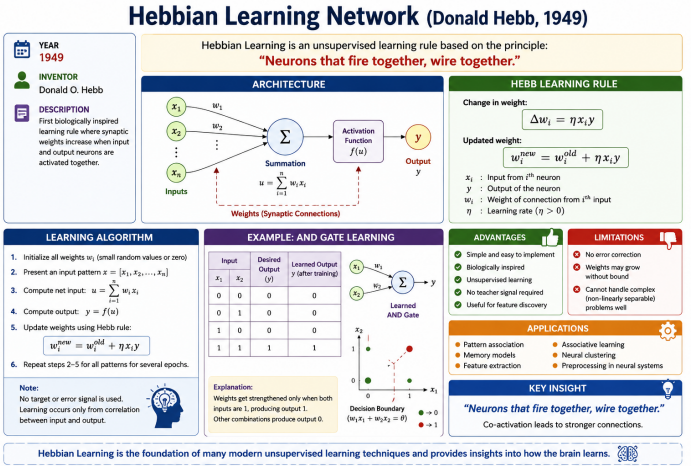
McCulloch–Pitts (MCP) Neuron



Key Takeaway

The McCulloch–Pitts (MCP) Neuron, proposed in 1943, is the first mathematical model of an artificial neuron. It uses binary inputs and a threshold activation function to perform logical decision-making.

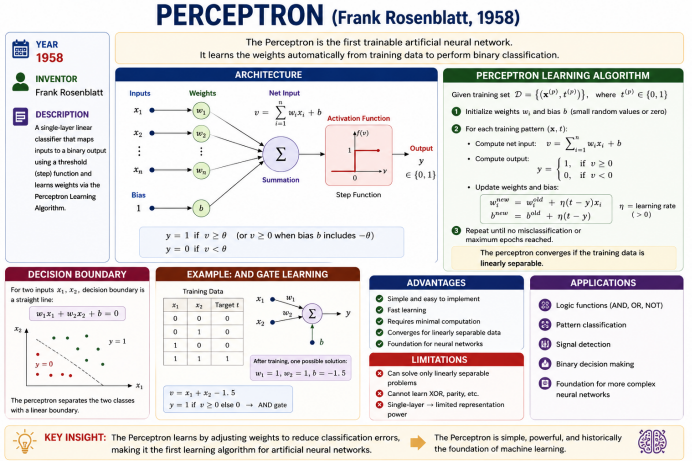
Hebbian Learning Network (Donald Hebb, 1949)



Key Takeaway

Hebbian Learning is the first biologically inspired learning rule in Artificial Intelligence. It strengthens the connection between neurons that activate together, making it the foundation of modern unsupervised learning.

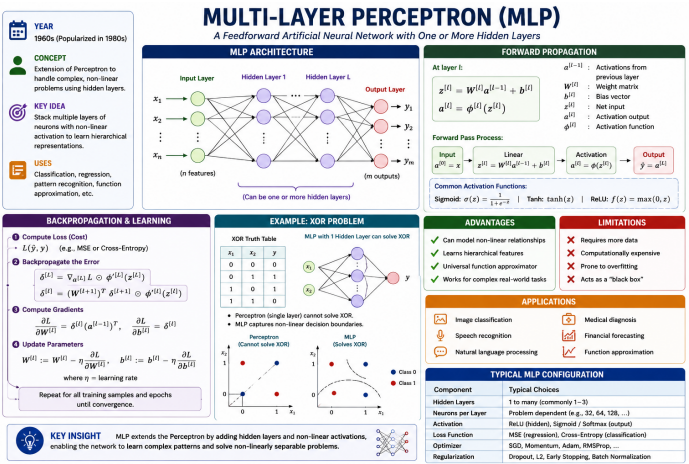
Perceptron (Frank Rosenblatt, 1958)



Key Takeaway

The Perceptron is the first trainable artificial neural network. It automatically learns weights from training data using the Perceptron Learning Algorithm and performs binary classification for linearly separable problems.

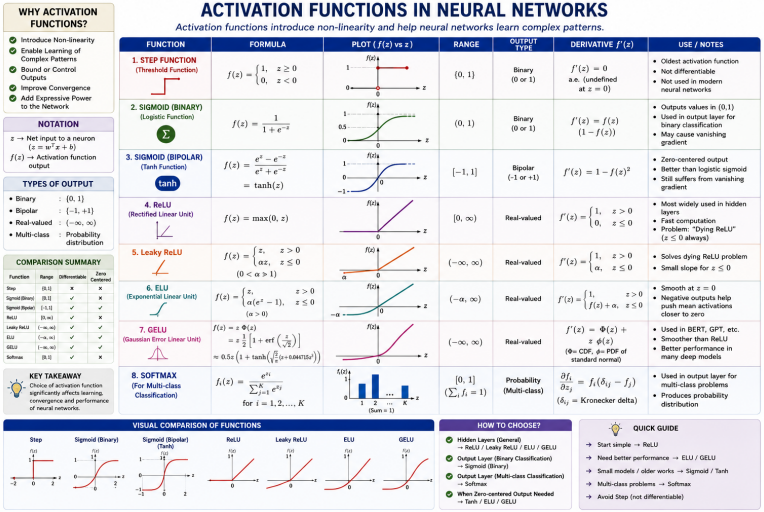
Multi-Layer Perceptron (MLP)



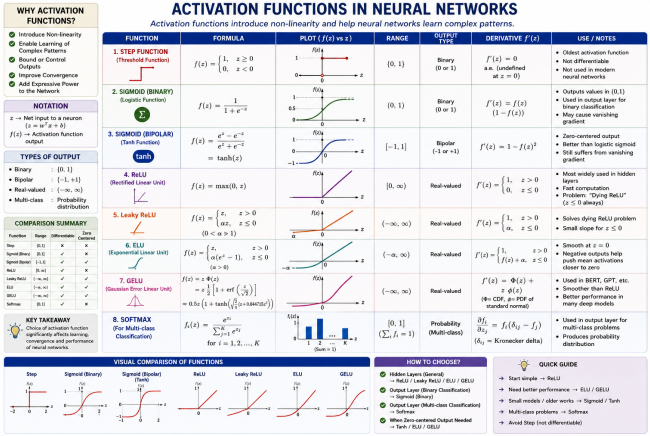
Key Takeaway

The Multi-Layer Perceptron (MLP) extends the Perceptron by introducing one or more hidden layers and nonlinear activation functions.
MLPs can learn complex, nonlinear decision boundaries and are trained using the Backpropagation algorithm.

Activation Functions in Neural Networks



Activation Functions in Neural Networks



Key Takeaway

Activation functions introduce non-linearity, enabling neural networks to learn complex patterns.
ReLU-family functions are common in hidden layers, while Softmax is widely used for multiclass outputs.

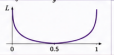
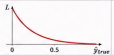
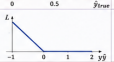
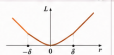
Common Activation Functions

- Step
- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- ELU
- GELU
- Softmax

Loss Functions in Neural Networks

LOSS FUNCTIONS IN NEURAL NETWORKS

A loss function (cost function) measures the difference between the predicted output and the true target.

Loss Function	Type / Use	Formula $L(y, \hat{y})$	Where	Plot L vs. $\hat{y}$ (for fixed y)	Range	Key Properties
1. Mean Squared Error (MSE) (L2 Loss)	Regression	$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	y : True value $\hat{y}$: Predicted value n : Number of samples		$[0, \infty)$	• Penalizes large errors heavily • Sensitive to outliers • Differentiable everywhere
2. Mean Absolute Error (MAE) (L1 Loss)	Regression	$L = \frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$			$[0, \infty)$	• Less sensitive to outliers • Linear penalty • Not differentiable at $y = \hat{y}$
3. Binary Cross-Entropy (BCE)	Binary Classification	$L = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$	$y \in \{0, 1\}$ $\hat{y} \in (0, 1)$ (probability)		$[0, \infty)$	• Used with Sigmoid output • Penalizes confident wrong predictions heavily • Widely used in binary tasks
4. Categorical Cross-Entropy (Softmax Loss)	Multi-class Classification	$L = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$	$y_k \in \{0, 1\}$ (one-hot) $\hat{y}_k$ (softmax prob.)		$[0, \infty)$	• Used with Softmax output • Measures how well the model predicts the correct class • Standard loss for multi-class
5. Hinge Loss (SVM Loss)	Binary Classification	$L = \frac{1}{n}\sum_{i=1}^{n}\max(0, 1 - y_i\hat{y}_i)$ where $y_i \in \{-1, +1\}$	$y_i \in \{-1, +1\}$ $\hat{y}_i$ (score / margin)		$[0, \infty)$	• Used in SVMs • Maximizes margin • Not differentiable at 1
6. Huber Loss	Regression (Robust)	$L_\delta(r) = \begin{cases} \frac{1}{2}r^2, & \lvert r\rvert \le \delta \\ \delta(\lvert r\rvert - \frac{1}{2}\delta), & \lvert r\rvert > \delta \end{cases}$ where $r = y - \hat{y}$			$[0, \infty)$	• Combines MSE and MAE • Robust to outliers • Differentiable everywhere

WHEN TO USE WHAT?

- **MSE** — Use for regression when errors are small and outliers are not a big issue.
- **MAE** — Use for regression when you want robustness to outliers.
- **BCE** — Use for binary classification with Sigmoid output.
- **Categorical CE** — Use for multi-class classification with Softmax output.
- **Hinge Loss** — Use for maximum margin classification (SVM).
- **Huber Loss** — Use for regression when you have outliers but still want smooth gradients.

COMMON NOTATION

y	True target value (label)
$\hat{y}$	Predicted value (output)
n	Number of samples
K	Number of classes
L	Loss (cost)
log	Natural logarithm

IMPORTANT NOTES

- The goal of training is to minimize the loss function.
- Lower loss → better predictions.
- Loss functions should be differentiable (or sub-differentiable) for gradient-based optimization.
- Choice of loss function depends on the problem type and the nature of errors.

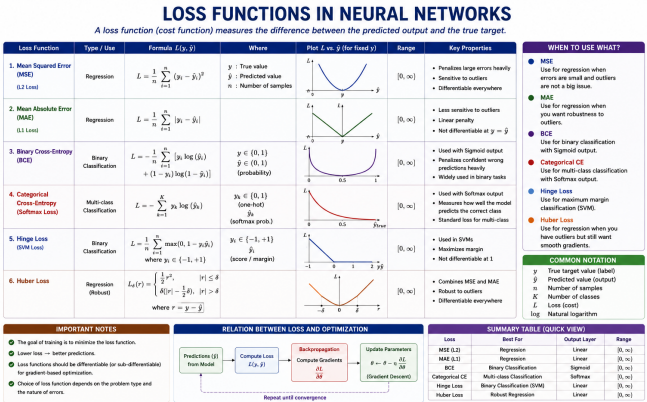
RELATION BETWEEN LOSS AND OPTIMIZATION

SUMMARY TABLE (QUICK VIEW)

Loss	Best For	Output Layer	Range
MSE (L2)	Regression	Linear	$[0, \infty)$
MAE (L1)	Regression	Linear	$[0, \infty)$
BCE	Binary Classification	Sigmoid	$[0, \infty)$
Categorical CE	Multi-class Classification	Softmax	$[0, \infty)$
Hinge Loss	Binary Classification (SVM)	Linear	$[0, \infty)$
Huber Loss	Robust Regression	Linear	$[0, \infty)$

Key Takeaway: Loss functions quantify how wrong the model is. Different losses suit different problems. The better the loss function matches the task, the faster and more effectively the model learns.

Loss Functions in Neural Networks



Key Takeaway

Loss functions quantify prediction error and provide the signal used to update model weights. Training minimizes the chosen loss; the best choice depends on the task and data.

Common Loss Functions

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Binary Cross-Entropy (BCE)
- Categorical Cross-Entropy
- Hinge Loss
- Huber Loss

DEEP LEARNING LAB

Course Code: 22CAC10N

Laboratory Course

Mr. Panigrahi Srikanth

Assistant Professor
Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)

Chaitanya Bharathi Institute of Technology (Autonomous)
Gandipet, Hyderabad–500075

B.E. CSE (AI & ML)

Academic Year 2026–2027

Deep Learning Laboratory Experiments

Part I – Foundations

1. Installation and Configuration of TensorFlow, PyTorch, and Keras; Perceptron Learning Algorithm for Boolean Logic Operations.
2. Multilayer Perceptron (MLP) for Classification and Regression.
3. Optimization Algorithms: Gradient Descent, Momentum, RMSProp, and Adam.
4. Regularization Techniques: L1/L2 Regularization, Dropout, Early Stopping, and Hyperparameter Tuning.

Laboratory Experiments

5. Implementation of a Convolutional Neural Network (CNN) for image classification.
6. Implementation of Pre-trained CNN Models (VGG16 and ResNet50) and comparison of their performance.
7. Implementation of a Long Short-Term Memory (LSTM) Network for next-word prediction using sequential text data.
8. Implementation of an Encoder–Decoder Model with an Attention Mechanism for neural machine translation.

Part III – Advanced Deep Learning

Laboratory Experiments

9. Implementation and Fine-Tuning of a BERT Model for sentiment analysis.

10. Implementation of a Generative Adversarial Network (GAN) for synthetic image generation.

Expected Learning Outcomes

Students will gain practical experience in implementing state-of-the-art deep learning models for Natural Language Processing (NLP), Computer Vision, and Generative Artificial Intelligence applications.

Installation and Configuration of Deep Learning Frameworks

TensorFlow • PyTorch • Keras

Implementation of the Perceptron Learning Algorithm

Boolean Logic Operations (AND, OR, NOT)

Experiment Objectives

After completing this experiment, students will be able to:

1. Install and configure TensorFlow, PyTorch, and Keras.
2. Verify Python environment and GPU availability.
3. Understand the Perceptron learning algorithm.
4. Implement Boolean logic operations using a single-layer perceptron.
5. Analyze perceptron learning and decision boundaries.

Software and Hardware Requirements

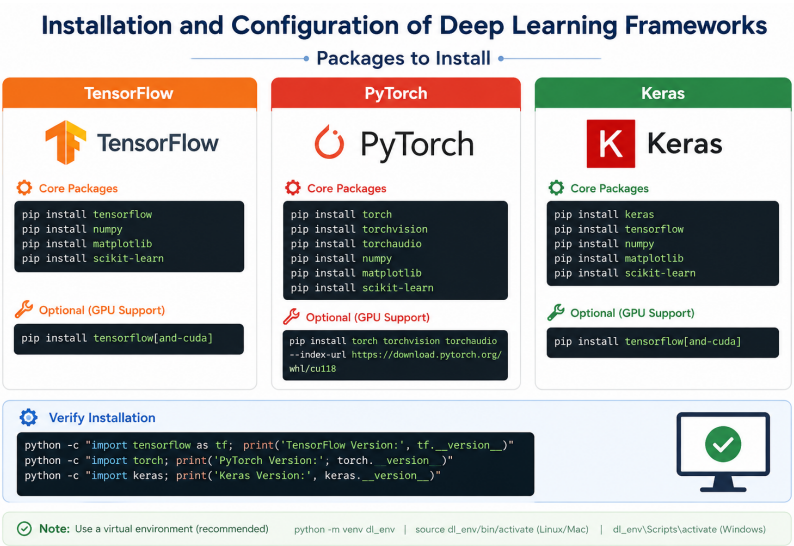
Software

- Python 3.10+
- Jupyter Notebook
- Google Colab
- TensorFlow
- PyTorch
- Keras

Hardware

- Windows/Linux/macOS
- Minimum 8 GB RAM
- NVIDIA GPU (Optional)
- CUDA Toolkit (Optional)
- Internet Connection

Installation and Configuration of Deep Learning Frameworks

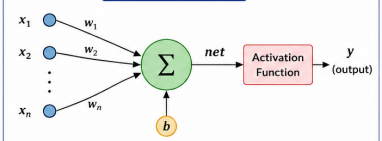


Perceptron Learning Algorithm

Perceptron Learning Algorithm

A Perceptron is a **single-layer linear classifier** used for binary classification problems.

Perceptron Model



Inputs: $x = [x_1, x_2, \ldots, x_n]^T$

Weights: $w = [w_1, w_2, \ldots, w_n]^T$

Bias: b

Mathematical Formulation

- **Net input (weighted sum):**

$$net = \sum_{i=1}^{n} w_i x_i + b = w^T x + b$$

- **Activation function (Step function):**

$$y = f(net) = \begin{cases} 1, & \text{if } net \geq 0 \\ 0, & \text{if } net < 0 \end{cases}$$

- **Error:** $e = t - y$

$t, y \in \{0, 1\}$ **for Binary Classification**

Where,

- x_i → inputs
- w_i → weights
- b → bias
- y → predicted output
- t → target output
- η → learning rate

Perceptron Learning Algorithm

1. Initialize weights (w_i) and bias (b) to small random values.
2. For each training example (x, t):
   - Compute $net = w^T x + b$
   - Compute output $y = f(net)$
   - Compute error $e = t - y$
3. Update the weights and bias:

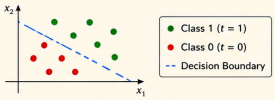
$$w_i(\text{new}) = w_i(\text{old}) + \eta\, e\, x_i \quad \text{for } i = 1, 2, \ldots, n$$

$$b(\text{new}) = b(\text{old}) + \eta\, e$$

4. Repeat steps ② and ③ until no error or maximum number of epochs is reached.

Key Points

- ★ Perceptron converges if the data is linearly separable.
- ★ It finds a linear decision boundary.
- ★ It cannot solve non-linearly separable problems (e.g., XOR).
- ★ Simple, fast and forms the basis of Neural Networks.



Truth Tables for Boolean Logic Operations

AND Gate

x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

x	Output
0	1
1	0

Boolean Logic Operations using Perceptron

Logic Gates Implemented

- **AND Gate**
  - Outputs 1 only when both inputs equal 1.
  - Demonstrates a linearly separable classification problem.
  - Perceptron successfully learns the AND decision boundary.
- **OR Gate**
  - Outputs 1 when at least one input equals 1.
  - Another linearly separable problem.
  - Perceptron converges to an optimal decision boundary.
- **NOT Gate**
  - Unary Boolean operation with a single input.
  - Output is the logical complement of the input.
  - Demonstrates Perceptron classification using one input feature.

Boolean Logic Operations using Perceptron

Concept

A single-layer Perceptron is a binary linear classifier that learns a decision boundary by adjusting weights and bias. It can successfully implement linearly separable Boolean logic operations such as AND, OR, and NOT gates.

The Perceptron computes

$$net = w_1 x_1 + w_2 x_2 + b$$

and produces the output using the step activation function

$$y = \begin{cases} 1, & net \geq 0 \\ 0, & net < 0 \end{cases}$$

Mathematical Representation of Boolean Gates

Logic Gate	Weights (w_1, w_2)	Bias (b)	Decision Function
AND	$(1, 1)$	-1.5	$y = H(x_1 + x_2 - 1.5)$
OR	$(1, 1)$	-0.5	$y = H(x_1 + x_2 - 0.5)$
NOT	(-1)	0.5	$y = H(-x + 0.5)$

where

$$H(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

is the Step Activation Function.

Truth Tables and Perceptron Computation

AND Gate

$net = x_1 + x_2 - 1.5$

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

$net = x_1 + x_2 - 0.5$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

$net = -x + 0.5$

x	y
0	1
1	0

Experiment 1 – Problem Statement

Objective

Develop single-layer perceptron models that learn AND, OR, and NOT operations by adjusting their weights and bias from labeled Boolean examples.

Learning Goals

- Understand binary classification.
- Apply supervised learning.
- Train perceptron models.
- Analyze decision boundaries.
- Evaluate prediction accuracy.

Problem Description

Given binary inputs $x_1, x_2 \in \{0, 1\}$, train a perceptron to predict $y \in \{0, 1\}$ for the target Boolean operation.
The implementation must:

- prepare the truth-table training data;
- train separate AND, OR, and NOT models;
- record learned weights and bias;
- compare predictions with target outputs.

Success Criterion

Every truth-table row must be classified correctly after training.

Perceptron Model and Learning Rule

Linear Model

For input vector $\mathbf{x} = [x_1, x_2]^\top$,

$$z = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + w_2 x_2 + b.$$

- $\mathbf{w} = [w_1, w_2]^\top$ contains trainable weights.
- b is the trainable bias.
- z is the linear decision score.

Step Activation

$$\hat{y} = H(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

Training Data and Error

For the labeled dataset $D = \{(\mathbf{x}_i, y_i) \mid i = 1, \ldots, N\}$, compute a prediction $\hat{y}_i$ and the error $e_i = y_i - \hat{y}_i$.

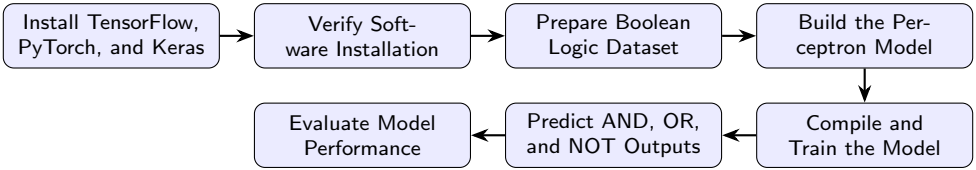
Perceptron Update

For learning rate $\eta > 0$, update

$$\mathbf{w} \leftarrow \mathbf{w} + \eta e_i \mathbf{x}_i, \qquad b \leftarrow b + \eta e_i.$$

Repeat until all samples are classified correctly or the iteration limit is reached. Because AND, OR, and NOT are linearly separable, convergence to a valid boundary is expected.

Experimental Workflow



Outcome

The Perceptron model learns the decision boundary for linearly separable Boolean logic operations and predicts binary outputs accurately.

Implementation Steps (Notebook Overview)

Implementation Pipeline

1. Import the required Python libraries.
2. Create the Boolean logic datasets (AND, OR, and NOT).
3. Visualize the input data distribution.
4. Build the Perceptron model using TensorFlow/Keras.
5. Compile the model with the SGD optimizer and Binary Cross-Entropy loss.
6. Train the model using the Boolean datasets.
7. Predict the output classes for unseen inputs.
8. Evaluate the model using performance metrics.
9. Visualize the decision boundary and learning curves.
10. Analyze the learned weights and bias values.

Experimental Results

Model Performance

The Perceptron model was successfully trained using the Boolean logic datasets (AND, OR, and NOT). The model learned the linear decision boundary and correctly classified the input patterns.

Observations

- Successful convergence during training.
- High classification accuracy for AND, OR, and NOT gates.
- Correctly learned the optimal weights and bias.
- Decision boundary separates the two classes effectively.

Performance Evaluation

- Accuracy
- Precision
- Recall
- F1-Score
- Matthews Correlation Coefficient (MCC)
- Confusion Matrix

Conclusion and Learning Outcomes

Conclusion

- Successfully installed and configured TensorFlow, PyTorch, and Keras.
- Implemented the Perceptron Learning Algorithm for Boolean logic operations.
- Trained the model using linearly separable datasets (AND, OR, and NOT).
- Evaluated the model using standard classification performance metrics.
- Observed that a single-layer perceptron correctly classifies linear problems but cannot solve the XOR problem.

Learning Outcomes

After completing this experiment, students will be able to:

- Configure deep learning development environments.
- Build and train a single-layer Perceptron model.
- Understand weight updates and the learning process.

Experiment Overview

Experiment Title

Implementation of a Multilayer Perceptron (MLP) for Binary Classification using the Breast Cancer Wisconsin Dataset.

Objectives

- Understand the architecture of a Multilayer Perceptron (MLP).
- Load and preprocess the Breast Cancer Wisconsin dataset.
- Train an MLP model for binary classification.
- Evaluate the model using standard performance metrics.

Multilayer Perceptron (MLP)

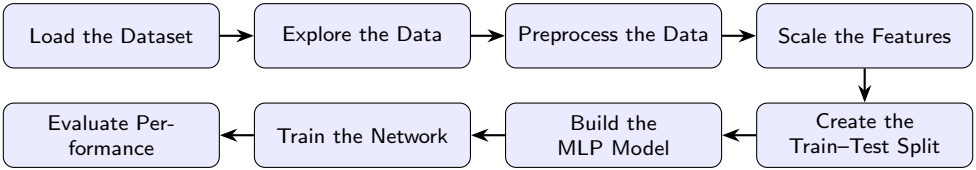
Overview

A Multilayer Perceptron (MLP) is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. The hidden neurons learn complex nonlinear relationships using activation functions.

MLP Components

- Input Layer
- Hidden Layer(s)
- Output Layer
- Weights and Biases
- Activation Functions (ReLU, Sigmoid)

Experimental Workflow



Dataset

Breast Cancer Wisconsin Diagnostic Dataset for binary classification.

Implementation Steps (Notebook Overview)

Implementation Pipeline

1. Import required Python libraries.
2. Load the Breast Cancer Wisconsin dataset.
3. Perform exploratory data analysis (EDA).
4. Preprocess and standardize the dataset.
5. Split the data into training and testing sets.
6. Build the MLP model using TensorFlow/Keras.
7. Train the network.
8. Predict the test samples.
9. Evaluate the model using performance metrics.
10. Visualize learning curves and confusion matrix.

Note

Experimental Results

Performance Metrics

- Accuracy
- Precision
- Recall
- F1-Score
- MCC
- ROC-AUC
- Cohen's Kappa

Observations

- High classification accuracy.
- Stable convergence during training.
- Low validation loss.
- Good generalization on test data.

Refer to the notebook for detailed plots, confusion matrix, ROC curve, and statistical validation.

Conclusion and Learning Outcomes

Conclusion

- Successfully implemented an MLP model for binary classification.
- Applied preprocessing and feature scaling techniques.
- Trained and evaluated the network using the Breast Cancer dataset.
- Achieved reliable classification performance using deep learning.

Learning Outcomes

Students will be able to:

- Understand the architecture of an MLP.
- Build neural networks using TensorFlow/Keras.
- Apply preprocessing techniques.
- Evaluate classification models using standard metrics.
- Interpret deep learning results and visualizations.